

AWS SageMaker Training

01-ml-infrastructure/sagemaker/training

Overview

SageMaker Training runs managed training jobs on ephemeral infrastructure. You provide code, data locations, and resource configuration, and SageMaker provisions the instances, streams logs, stores artifacts, and tears the instances down when the job completes.

This is the core managed primitive for repeatable training because it separates model code from the underlying host lifecycle and makes experiments easier to automate.

Core Concepts

- A training job is defined by the training image, entry point script, input channels, output path, instance count, instance type, and optional distributed training settings.
- Data can be delivered through multiple modes such as file, fast file, or pipe depending on access patterns and startup needs.
- Managed spot training, checkpoints, profiler tooling, and metrics extraction can reduce cost and improve visibility for long or expensive jobs.

How It Works

- The training script is packaged with framework specific estimators or custom containers and launched with a role that can access the required data and artifact locations.
- SageMaker downloads or streams the data, executes the container, captures logs to CloudWatch, and writes the model artifact and any generated metrics back to S3.
- A completed job can feed later stages such as model evaluation, registration, batch transform, or endpoint deployment.

AI and ML Relevance

- Managed training is well suited for scheduled retraining, hyperparameter tuning, and experiments that need clean, isolated, reproducible execution environments.
- Distributed training support matters when model size or dataset size outgrows a single node, especially for deep learning workloads on GPU fleets.

Operational Notes

- Checkpointing is essential when using spot instances or any training run that would be expensive to restart from zero.
- Track both infrastructure metrics and model metrics. Good GPU utilization with bad validation loss still means the experiment failed.
- Keep containers lean and training inputs explicit so runs are reproducible and startup times do not become an invisible source of waste.

What To Remember

- Training jobs are ephemeral by design. Persistent state belongs in S3 or attached managed storage, not only inside the container filesystem.
- Input channels define what the job sees; artifact output defines what the rest of the platform can reuse.
- If you cannot reproduce a training run from code, data version, parameters, and image version, the platform is not mature enough.